Unit V

JAVAFX EVENT HANDLING, CONTROLS & COMPONENTS FOR GUI PROGRAM

JAVAFX Events and Controls: Event Basics – Handling Key and Mouse Events, Controls: Checkbox, ToggleButton – RadioButtons – ListView – ComboBox – ChoiceBox – TextBox – ScrollPane. Layouts – Flowpane – HBox and VBox – BorderPane – StackPane – GridPane. Menus –Basics – Menu – Menu bars – Menu Item.

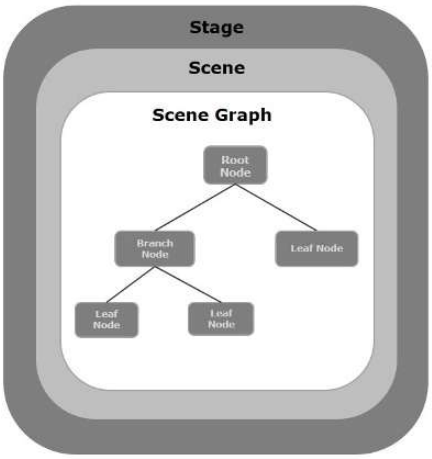
5.1 Basics of JavaFX

- JavaFX is a set of graphics and media packages that enable developers to design, create, test, debug, and deploy desktop applications and Rich Internet Applications (RIA) that operate consistently across diverse platforms.
- JavaFX makes it easier to create desktop applications and games in java

5.1.2 Features of JavaFX

- JavaFX library is written in Java. Hence developer find it easy to learn and write the applications in JavaFX
- JavaFX library creates UI controls using which any GUI based application can be developed
- It provides the classes for 2D and 3D graphics
- It contains a set of ready-to-use chart components, so you don't have to code that from scratch every time you need a basic chart
- JavaFX components can be styled using CSS
- It provides the support for integrating audio, videos and web applications

5.1.3 JavaFX Application Structure



- three major components

1) Stage
   - ✓ Stage(a window) in a JavaFX application is similar to the Frame in a Swing Application. It acts like a container for all the JavaFX objects.
   - ✓ PrimaryStage is created internally by the platform. Other stages can further be created by the application.
   - ✓ A stage has two parameters determining its position namely **Width** and **Height**. It is divided as Content Area and Decorations (Title Bar and Borders).
   - ✓ There are five types of stages available −
     - o Decorated
     - o Undecorated
     - o Transparent
     - o Unified
     - o Utility
   - ✓ We have to call the **show()** method to display the contents of a stage.

2) Scene
   - ✓ A scene represents the physical contents of a JavaFX application. It contains all the contents of a scene graph.
   - ✓ The class Scene of the package javafx.scene represents the scene object. At an instance, the scene object is added to only one stage.

3) Nodes
   - ✓ A scene graph is a tree-like data structure (hierarchical) representing the contents of a scene. In contrast, a node is a visual/graphical object of a scene graph.
   - ✓ A node may include −
     - **Geometrical** (Graphical) objects (2D and 3D) such as − Circle, Rectangle, Polygon, etc.
     - **UI Controls** such as − Button, Checkbox, Choice Box, Text Area, etc.
     - **Containers** (Layout Panes) such as Border Pane, Grid Pane, Flow Pane, etc.
     - **Media elements** such as Audio, Video and Image Objects.

✓ A node is of three types −

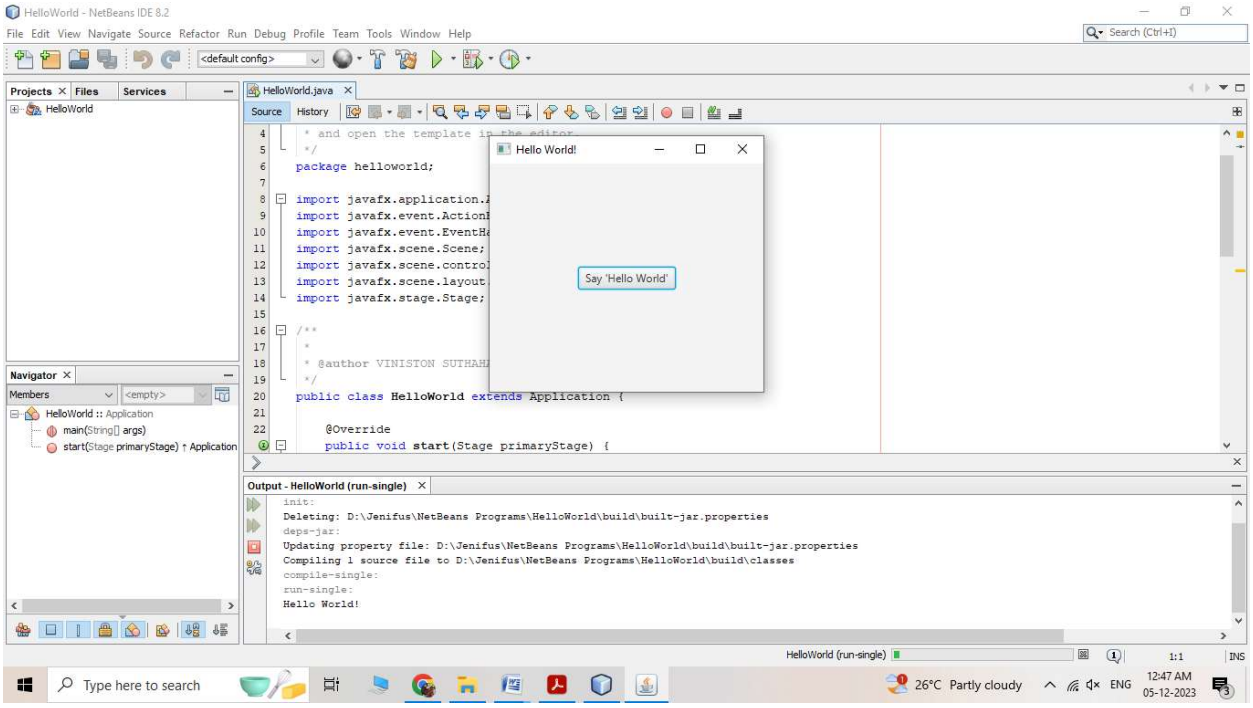
1. **Root Node** − The first Scene Graph is known as the Root node.
2. **Branch Node/Parent Node** − the node with child nodes are known as branch/parent nodes. The parent nodes will be of the following types −
   - **Group** − A group node is a collective node that contains a list of children nodes. Whenever the group node is rendered, all its child nodes are rendered in order. Any transformation, effect state applied on the group will be applied to all the child nodes.
   - **Region** − It is the base class of all the JavaFX Node based UI Controls, such as Chart, Pane and Control.
   - **WebView** − This node manages the web engine and displays its contents.
3. **Leaf Node** − The node without child nodes is known as the leaf node.

5.1.4 General Rules for writing JavaFX Application:

- A JavaFX Application must extend javafx.application.Application.
- The main() method should call Application.launch()
- The start() method is the main entry point for all JavaFX applications
  - start() is called when a Stage is connected to the Operating System's window
- The content of the scene is represented as a hierarchical scene graph of nodes:
  - Stage is the top-level JavaFX Container
  - Scene is the container for all content

5.1.5 Writing First JavaFX Application Program

1. From **File** menu, choose **New Project.** Then select **JavaFX** application category choose **JavaFX Application**. Click **Next**.
2. Name the project **HelloWorld** and click **Finish**.
3. Code will get automatically generated
4. From **Run** Menu, choose **Run File**



import javafx.application.Application - package to support the Application class

```
import javafx.event.ActionEvent;
import javafx.event.EventHandler;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;
```

} Classes for supporting Stage Scene, Button component and launch method

public class HelloWorld extends Application{ - Class must extend **Application** class

```
Button btn = new Button();
btn.setText("Say 'Hello World'");
```

} Creating Button Component

Scene scene = new Scene(root, 300, 250); - create scene using new operator and place button component on scene

primaryStage.setScene(scene); - set scene for stage

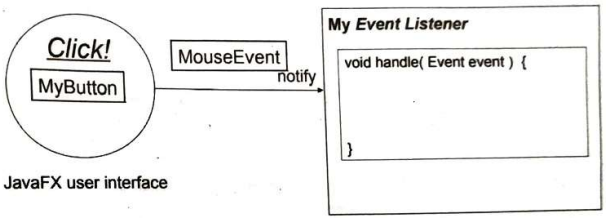
primaryStage.show(); - component can be made visible

```
public static void main(String[] args) {
      launch(args);
   }
```
Call launch() method to call start() method

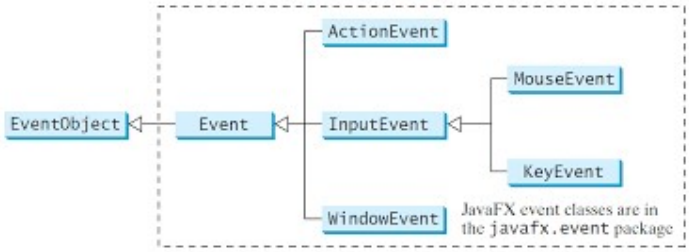
5.2 JAVAFX Events

- Any activity that interrupts the current ongoing activity.
- Ex. When user clicks a button, then an event is generated. Code is to be written for processing button click action

Event Source – object that generates event. Ex. Mouse
Event Handler – code written to process the generated event
Event Listener – task of handling an event is carried out by event listener. When an event occurs, an **event object** is created first. This object is then passed to a **Listener.** A listener **implements the interface** that has the method for event handling



Event Class Hierarchy



5.2.1 Handling Mouse Event

Mouse event is fired when the mouse button in pressed, released, clicked, moved or dragged on the node

EventHandler Property	Description	Setter Methods
onMouseClicked	This is of the type EventHandler of MouseEvent. This is assigned to a function which is to be called when the mouse button is clicked on the node.	setOnMouseClicked(EventHandler value)
onMouseDragEntered	This is of the type EventHandler of MouseDragEvent. This is assigned to a function which is to be called when a full press drag release gesture enters the node.	setOnMouseDragEntered(EventHandler value)
onMouseDragExited	This is of the type EventHandler of MouseDragEvent. This is assigned to a function which is to be called when a full press drag release gesture exits the node.	setOnMouseDragExited(EventHandler value)
onMouseDragged	This is of the type EventHandler of MouseDragEvent. This is assigned to a function which is to be called when the mouse button is pressed and dragged on the node.	setOnMouseDragged(EventHandler value)
onMouseMoved	This is of the type EventHandler of MouseEvent. This is assigned to a function which is to be called when the mouse moves within the node and no button has been pushed.	setOnMouseMoved(EventHandler value)
onMousePressed	This is of the type EventHandler of MouseEvent. This is assigned to a function which is to be called when the mouse button is pressed on the node.	setOnMousePressed(EventHandler value)

onMouseReleased	This is of the type EventHandler of MouseEvent. This is assigned to a function which is to be called when the mouse button is released on the node.	setOnMouseReleased(EventHandler value)

Methods for MouseEvent

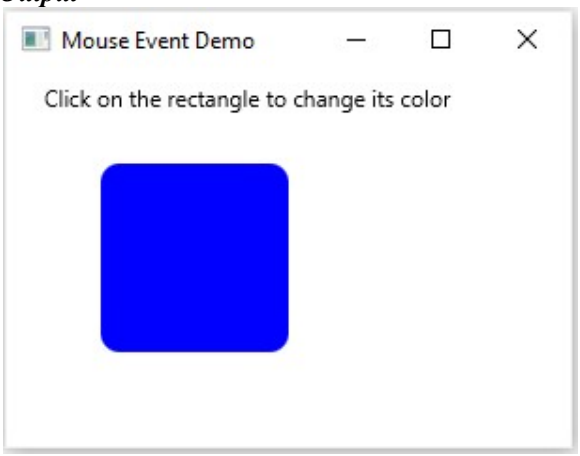
Method	Description
MouseButton getButton()	It represents which mouse button is pressed.
int getClickCount()	It returns the number of mouse clicks.
double getX()	It returns the x-coordinate of the mouse point in the event source node.
double getY()	It returns the y-coordinate of the mouse point in the event source node.
double getSceneX()	It returns the x-coordinate of the mouse point in the scene.
double getSceneY()	It returns the y-coordinate of the mouse point in the scene.
double getScreenX()	It returns the x-coordinate of the mouse point in the screen.
double getScreenY()	It returns the y-coordinate of the mouse point in the screen.
boolean isAltDown()	It checks if the ALT key is pressed or not.
boolean isControlDown()	It checks if the Control key is pressed or not.
boolean isShiftDown()	It checks if the Shidt key is pressed or not.

Example:

```
package myjavafxapplication;
import javafx.application.Application;
import javafx.event.EventHandler;
import javafx.scene.Scene;
import javafx.scene.Group;
import javafx.scene.input.MouseEvent;
import javafx.scene.paint.Color;
import javafx.scene.shape.Rectangle;
import javafx.scene.text.Text;
import javafx.stage.Stage;
public class myjavafxapplication extends Application {
public void start(Stage primaryStage) {
Rectangle rect=new Rectangle(50,50,100,100);
rect.setArcWidth(20);
rect.setArcHeight(20);
rect.setFill(Color.BLUE);
Text text=new Text(20,20,"Click on the rectangle to change its color");
rect.setOnMouseClicked(e->{
rect.setFill(Color.RED);
});
Group root=new Group(rect,text);
Scene scene=new Scene(root,300,200);
primaryStage.setScene(scene);
primaryStage.setTitle("Mouse Event Demo");
primaryStage.show();
}
public static void main(String[] args) {
```

```
launch(args);
}
}
```

Output



5.2.2 Handling Keyboard Event

When any key on the keyboard is pressed, released or typed on a node then the keyboard event occurs.

Event Registration Methods

User Action	Source Object	Event Type Fired	Event Registration Method
Key Pressed	Node, Scene	KeyEvent	setOnKeyPressed(EventHandler<KeyEvent>)
Key Released			setOnKeyReleased(EventHandler<KeyEvent>)
Key typed			setOnKeyTyped(EventHandler<KeyEvent>)

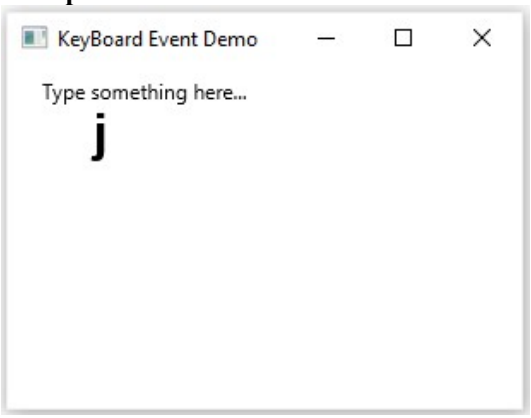
For recognizing the keyboard events we use **KeyEvent** class. Various methods of KeyEvent class are,

Method	Description
String getCharacter()	Returns the character associated with the key pressed.
KeyCode getCode()	Returns the key code associated with the key in the event.
String getText()	Returns the string describing the key code.
boolean isAltDown()	It returns true if the ALT key is down.
boolean isControlDown()	It returns true if the Control key is down.
boolean isMetaDown()	It returns true if the Meta button is pressed.
boolean isShiftDown()	It returns true if the shift key is pressed.

The key codes returned by the getCode() method are constants. The constants are,

Constant	Associated Key
HOME	The Home key
END	The End key
CONTROL	The control key
SHIFT	The shift key
PAGE_UP	The page up key
PAGE_DOWN	The page down key
UP	The Up key
DOWN	The Down key
LEFT	The left arrow key
RIGHT	The right arrow key
BACK_SPACE	The backspace tree
CAPS	The caps lock key
ESCAPE	The escape key
ENTER	The enter key
TAB	The tab key
NUM_LOCK	The num lock key
0 to 0	The keys from 0 to 9
A to Z	The letter keys from A to Z

Example:



5.3 Controls

- GUI is an essential part of any window based application
- Interaction between user and the application program is possible with the help of user interface (UI) controls that placed on the screen
- JavaFX allows its developer a rich set of UI controls. Some of the most commonly used UI Controls are enlisted in the following table

S.No.	Control	Description
1	Label	Label is a component that is used to define a simple text on the screen.
2	Button	Button is a component that controls the function of the application.
3	RadioButton	The Radio Button is used to provide various options to the user. The user can only choose one option among all.
4	CheckBox	Check Box is used to get the kind of information from the user which contains various choices.
5	TextField	Text Field is basically used to get the input from the user in the form of text.
6	Textarea	This control allows the user to enter multiple line text
7	PasswordField	PasswordField is used to get the user's password. Whatever is typed in the passwordfield is not shown on the screen to anyone.
8	HyperLink	HyperLink are used to refer any of the webpage through your appication.
9	Slider	Slider is used to provide a pane of options to the user in a graphical form where the user needs to move a slider over the range of values to select one of them.

10	ProgressBar	Progress Bar is used to show the work progress to the user.
11	ProgressIndicator	Instead of showing the analogue progress to the user, it shows the digital progress so that the user may know the amount of work done in percentage.
12	ScrollBar	JavaFX Scroll Bar is used to provide a scroll bar to the user so that the user can scroll down the application pages.
13	Menu	JavaFX provides a Menu class to implement menus. Menu is the main component of any application.
14	ToolTip	JavaFX ToolTip is used to provide hint to the user about any component. It is mainly used to provide hints about the text fields or password fields being used in the application.

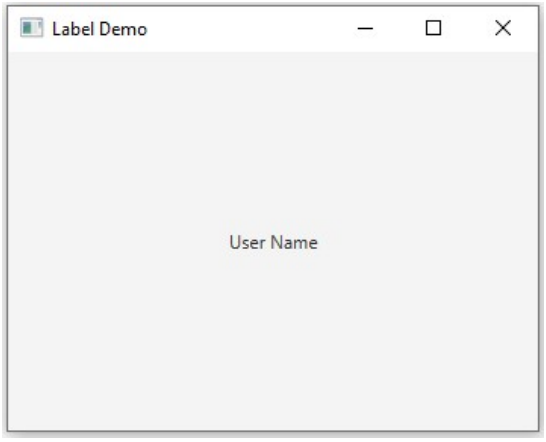
5.3.1 Label

- Displays simple text in the screen
- javafx.scene.control.Label need to be imported
- Constructors -
  - Label()
  - Label(String txt)
  - Label(String txt,Node graphics)

Example:

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.layout.StackPane;
import javafx.scene.control.Label;
import javafx.stage.Stage;
public class myjavafxapplication extends Application {
public void start(Stage stage) {
StackPane root=new StackPane();
Label L=new Label("User Name");
Scene scene=new Scene(root,350,250);
root.getChildren().add(L);
stage.setScene(scene);
stage.setTitle("Label Demo");
stage.show();
}
public static void main(String[] args) {
launch(args);
}
}
```

Output



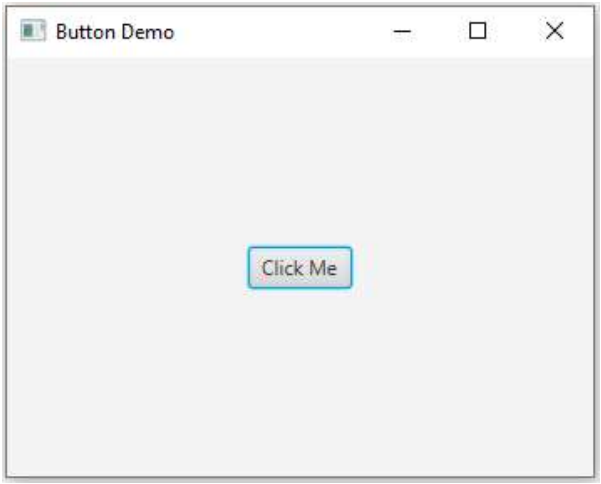
5.3.2 Button

- Controls the behaviour of the application
- Events get generated when a button is clicked
- Constructors -
  - Button(String txt)
  - Button()

Example:

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.layout.StackPane;
import javafx.scene.control.Button;
import javafx.stage.Stage;
public class myjavafxapplication extends Application {
public void start(Stage stage) {
StackPane root=new StackPane();
Button B=new Button("Click Me");
Scene scene=new Scene(root,350,250);
root.getChildren().add(B);
stage.setScene(scene);
stage.setTitle("Button Demo");
stage.show();
}
public static void main(String[] args) {
launch(args);
}
}
```

Output



5.3.3 Checkbox

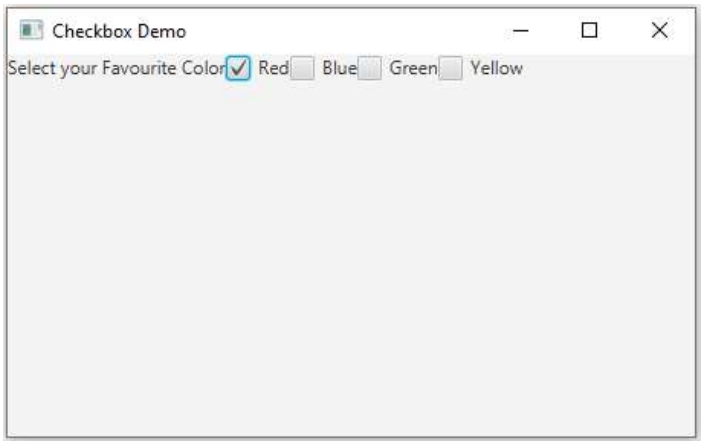
- Provide more than one choices at a time
- Constructors -
    - CheckBox()
    - CheckBox(String txt)

Example:

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.layout.HBox;
import javafx.scene.control.Label;
import javafx.scene.control.CheckBox;
import javafx.stage.Stage;
public class myjavafxapplication extends Application {
public void start(Stage stage) {
HBox root=new HBox();
Label L=new Label("Select your Favourite Color");
CheckBox CB1=new CheckBox("Red");
CheckBox CB2=new CheckBox("Blue");
CheckBox CB3=new CheckBox("Green");
CheckBox CB4=new CheckBox("Yellow");
Scene scene=new Scene(root,450,250);
root.getChildren().addAll(L,CB1,CB2,CB3,CB4);
stage.setScene(scene);
```

```
stage.setTitle("Checkbox Demo");
stage.show();
}
public static void main(String[] args) {
launch(args);
}
}
```

Output



5.3.4 ToggleButton

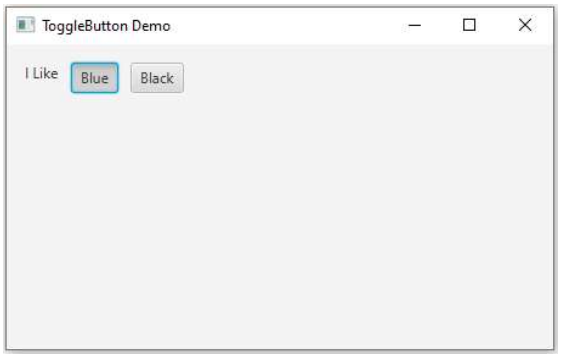
- Button control that has two states selected or deselected
- ToggleButtons are placed in a Toggle Group
- Only one button is selected at a time
- Constructors -
    - ToggleButton()
    - ToggleButton(String txt)

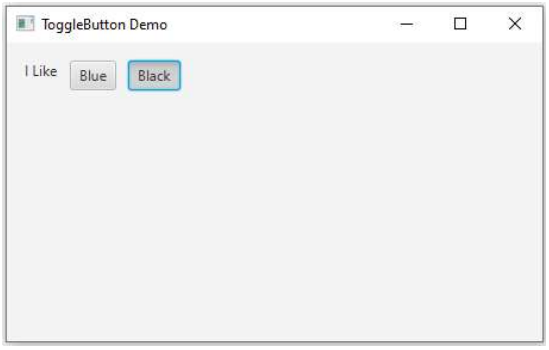
Example:

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.layout.HBox;
import javafx.geometry.Insets;
import javafx.scene.control.Label;
import javafx.scene.control.ToggleButton;
import javafx.scene.control.ToggleGroup;
import javafx.stage.Stage;
public class myjavafxapplication extends Application {
public void start(Stage stage) {
HBox root=new HBox();
Label L=new Label("I Like");
root.setPadding(new Insets(15));
root.setSpacing(10);
ToggleGroup group=new ToggleGroup();
ToggleButton TB1=new ToggleButton("Blue");
ToggleButton TB2=new ToggleButton("Black");
TB1.setToggleGroup(group);
TB2.setToggleGroup(group);
TB1.setSelected(true);
root.getChildren().addAll(L,TB1,TB2);
Scene scene=new Scene(root,450,250);
stage.setScene(scene);
stage.setTitle("ToggleButton Demo");
stage.show();
}
public static void main(String[] args) {
launch(args);
```

```
}
}
```

Output





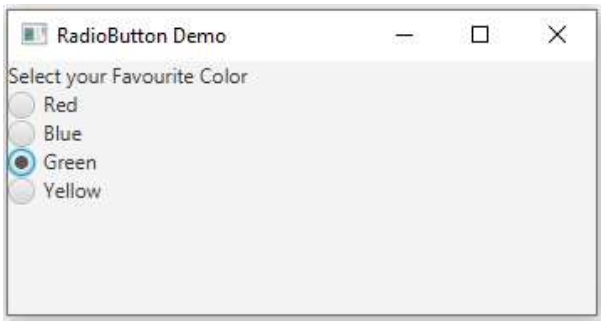
5.3.5 RadioButton

- Used to make a choice
- Can select only one option at a time
- Constructors -
  - RadioButton()
  - RadioButton(String txt)

Example:

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.layout.VBox;
import javafx.scene.control.Label;
import javafx.scene.control.RadioButton;
import javafx.scene.control.ToggleGroup;
import javafx.stage.Stage;
public class myjavafxapplication extends Application {
public void start(Stage stage) {
VBox root=new VBox();
Scene scene=new Scene(root,350,150);
ToggleGroup TG=new ToggleGroup();
Label L=new Label("Select your Favourite Color");
RadioButton RB1=new RadioButton("Red");
RadioButton RB2=new RadioButton("Blue");
RadioButton RB3=new RadioButton("Green");
RadioButton RB4=new RadioButton("Yellow");
RB1.setToggleGroup(TG);
RB2.setToggleGroup(TG);
RB3.setToggleGroup(TG);
RB4.setToggleGroup(TG);
root.getChildren().addAll(L,RB1,RB2,RB3,RB4);
stage.setScene(scene);
stage.setTitle("RadioButton Demo");
stage.show();
}
public static void main(String[] args) {
launch(args);
}
}
```

Output



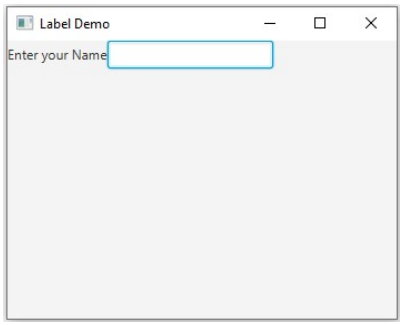
5.3.6 TextField

- Allows the user to enter the text that can be read by the application
- Constructors -
    - TextField()
    - TextField(String txt)

Example:

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.layout.GridPane;
import javafx.scene.control.Label;
import javafx.scene.control.TextField;
import javafx.stage.Stage;
public class myjavafxapplication extends Application {
public void start(Stage stage) {
GridPane root=new GridPane();
Scene scene=new Scene(root,350,250);
Label L=new Label("Enter your Name");
TextField TF=new TextField();
root.addRow(0,L,TF);
stage.setScene(scene);
stage.setTitle("Label Demo");
stage.show();
}
public static void main(String[] args) {
launch(args);
}
}
```

Output



5.3.7 TextArea

- Allows the user to enter multiple line text that can be read by the application
- Constructors -
    - TextArea()
    - TextArea(String txt)

Example:

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.layout.GridPane;
import javafx.scene.control.Label;
import javafx.scene.control.TextArea;
import javafx.stage.Stage;
public class myjavafxapplication extends Application {
public void start(Stage stage) {
GridPane root=new GridPane();
Scene scene=new Scene(root,650,250);
Label L=new Label("Enter your Address");
TextArea TA=new TextArea();
root.addRow(0,L,TA);
stage.setScene(scene);
stage.setTitle("TextArea Demo");
stage.show();
}
```

```
public static void main(String[] args) {
launch(args);
}
}
```

Output



5.3.8 ListView

- Enables user to choose one or more options from a predefined list of choices.
- Creating ListView

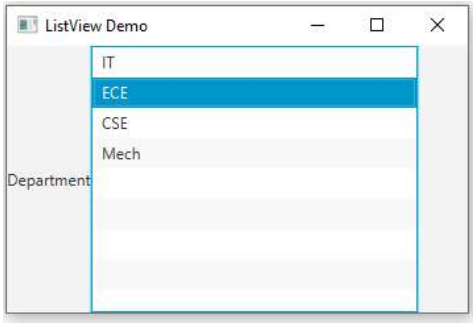
  ListView LV=new ListView();
- Adding items to ListView

  LV.getItems().add("Item1");

Example:

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.layout.GridPane;
import javafx.scene.control.Label;
import javafx.scene.control.ListView;
import javafx.scene.control.SelectionMode;
import javafx.stage.Stage;
public class myjavafxapplication extends Application {
public void start(Stage stage) {
GridPane root=new GridPane();
Scene scene=new Scene(root,350,250);
Label L=new Label("Department");
ListView LV=new ListView();
LV.getItems().add("IT");
LV.getItems().add("ECE");
LV.getItems().add("CSE");
LV.getItems().add("Mech");
root.addRow(0,L,LV);
stage.setScene(scene);
stage.setTitle("ListView Demo");
stage.show();
}
public static void main(String[] args) {
launch(args);
}
}
```

Output



5.3.9 ComboBox

- Enables user to choose one or more options from a predefined list of choices.
- Creating ComboBox

ComboBox CB=new ComboBox ();

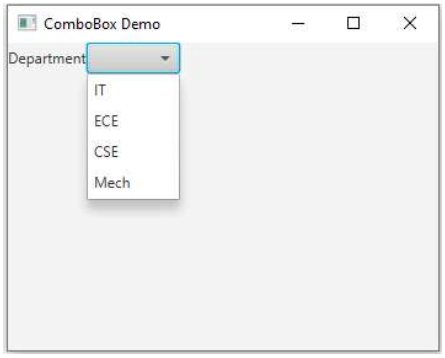
- Adding items to ComboBox

CB.getItems().add("Item1");

Example:

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.layout.GridPane;
import javafx.scene.control.Label;
import javafx.scene.control.ComboBox;
import javafx.scene.control.SelectionMode;
import javafx.stage.Stage;
public class myjavafxapplication extends Application {
public void start(Stage stage) {
GridPane root=new GridPane();
Scene scene=new Scene(root,350,250);
Label L=new Label("Department");
ComboBox CB=new ComboBox();
CB.getItems().add("IT");
CB.getItems().add("ECE");
CB.getItems().add("CSE");
CB.getItems().add("Mech");
root.addRow(0,L,CB);
stage.setScene(scene);
stage.setTitle("ComboBox Demo");
stage.show();
}
public static void main(String[] args) {
launch(args);
}
}
```

Output



5.3.10 ChoiceBox

- Enables user to choose only one option from a predefined list of choices.
- Creating ChoiceBox

ChoiceBox CB=new ChoiceBox ();

- Adding items to ChoiceBox

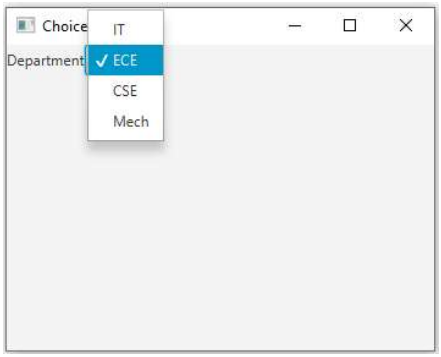
CB.getItems().add("Item1");

Example:

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.layout.GridPane;
import javafx.scene.control.Label;
import javafx.scene.control.ChoiceBox;
import javafx.scene.control.SelectionMode;
import javafx.stage.Stage;
public class myjavafxapplication extends Application {
public void start(Stage stage) {
GridPane root=new GridPane();
```

```
Scene scene=new Scene(root,350,250);
Label L=new Label("Department");
ChoiceBox CB=new ChoiceBox();
CB.getItems().add("IT");
CB.getItems().add("ECE");
CB.getItems().add("CSE");
CB.getItems().add("Mech");
root.addRow(0,L,CB);
stage.setScene(scene);
stage.setTitle("ChoiceBox Demo");
stage.show();
}
public static void main(String[] args) {
launch(args);
}
}
```

Output



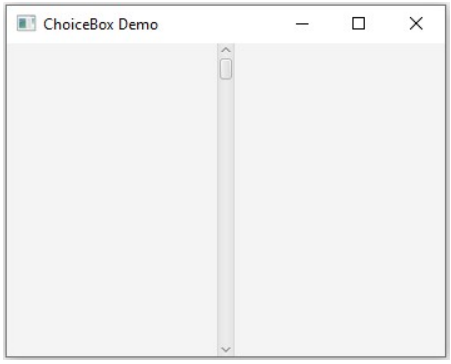
5.3.11 ScrollBar

- Enables user to scroll down in application page.
- Creating ScrollBar
  ScrollBar CB=new ScrollBar ();
- A horizontal scrollbar will be created by default
- To create Vertical scrollbar,
  SB.setOrientation(Orientation.VERTICAL);

Example:

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.layout.StackPane;
import javafx.scene.control.ScrollBar;
import javafx.geometry.Orientation;
import javafx.stage.Stage;
public class myjavafxapplication extends Application {
public void start(Stage stage) {
StackPane root=new StackPane();
Scene scene=new Scene(root,350,250);
ScrollBar SB=new ScrollBar();
SB.setOrientation(Orientation.VERTICAL);
root.getChildren().add(SB);
stage.setScene(scene);
stage.setTitle("ChoiceBox Demo");
stage.show();
}
public static void main(String[] args) {
launch(args);
}}
```

Output



5.4 Layouts

- The arrangement of various components in a scene within the container
- javafx.scene.layout Package provides various classes that represents the layouts

Class	Description
BorderPane	Organizes nodes in top, left, right, centre and the bottom of the screen.
FlowPane	Organizes the nodes in the horizontal rows according to the available horizontal spaces. Wraps the nodes to the next line if the horizontal space is less than the total width of the nodes
GridPane	Organizes the nodes in the form of rows and columns.
HBox	Organizes the nodes in a single row.
Pane	It is the base class for all the layout classes.
StackPane	Organizes nodes in the form of a stack i.e. one onto another
VBox	Organizes nodes in a vertical column.

5.4.1 FlowPane

Organizes the nodes in the horizontal rows according to the available horizontal spaces. Wraps the nodes to the next line if the horizontal space is less than the total width of the nodes

Example

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.layout.FlowPane;
import javafx.scene.control.Button;
import javafx.stage.Stage;
public class myjavafxapplication extends Application {
public void start(Stage stage) {
FlowPane root=new FlowPane();
Scene scene=new Scene(root,75,200);
Button B1=new Button("One");
Button B2=new Button("Two");
Button B3=new Button("Three");
Button B4=new Button("Four");
Button B5=new Button("Five");
Button B6=new Button("Six");
root.getChildren().addAll(B1,B2,B3,B4,B5,B6);
stage.setScene(scene);
stage.setTitle("FlowPane Demo");
stage.show();
}
public static void main(String[] args) {
launch(args);
}
}
```

Output



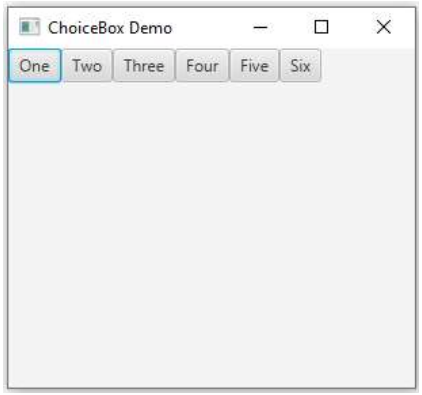
5.4.2 HBox

Organizes the nodes in the form of horizontal rows.

Example

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.layout.HBox;
import javafx.scene.control.Button;
import javafx.stage.Stage;
public class myjavafxapplication extends Application {
public void start(Stage stage) {
HBox root=new HBox();
Scene scene=new Scene(root,300,250);
Button B1=new Button("One");
Button B2=new Button("Two");
Button B3=new Button("Three");
Button B4=new Button("Four");
Button B5=new Button("Five");
Button B6=new Button("Six");
root.getChildren().addAll(B1,B2,B3,B4,B5,B6);
stage.setScene(scene);
stage.setTitle("ChoiceBox Demo");
stage.show();
}
public static void main(String[] args) {
launch(args);
}
}
```

Output



5.4.3 VBox

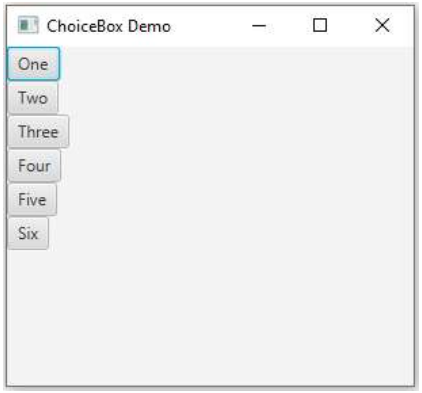
Organizes the nodes in the form of Vertical rows.

Example

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.layout.VBox;
import javafx.scene.control.Button;
import javafx.stage.Stage;
```

```
public class myjavafxapplication extends Application {
public void start(Stage stage) {
VBox root=new VBox();
Scene scene=new Scene(root,300,250);
Button B1=new Button("One");
Button B2=new Button("Two");
Button B3=new Button("Three");
Button B4=new Button("Four");
Button B5=new Button("Five");
Button B6=new Button("Six");
root.getChildren().addAll(B1,B2,B3,B4,B5,B6);
stage.setScene(scene);
stage.setTitle("ChoiceBox Demo");
stage.show();
}
public static void main(String[] args) {
launch(args);
}
}
```

Output



5.4.4 BorderPane

- Organizes the nodes in top, bottom, left, right and center positions.
- Methods:

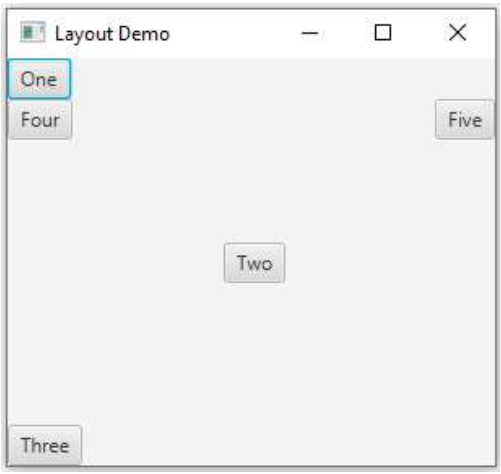
Setter Methods	Description
setBottom()	Add the node to the bottom of the screen
setCentre()	Add the node to the centre of the screen
setLeft()	Add the node to the left of the screen
setRight()	Add the node to the right of the screen
setTop()	Add the node to the top of the screen

Example

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.layout.BorderPane;
import javafx.scene.control.Button;
import javafx.stage.Stage;
public class myjavafxapplication extends Application {
public void start(Stage stage) {
BorderPane root=new BorderPane();
Scene scene=new Scene(root,300,250);
Button B1=new Button("One");
Button B2=new Button("Two");
Button B3=new Button("Three");
Button B4=new Button("Four");
```

```
Button B5=new Button("Five");
root.setTop(B1);
root.setCenter(B2);
root.setBottom(B3);
root.setLeft(B4);
root.setRight(B5);
stage.setScene(scene);
stage.setTitle("Layout Demo");
stage.show();
}
public static void main(String[] args) {
launch(args);
}
}
```

Output



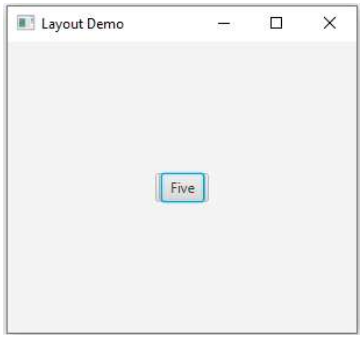
5.4.5 StackPane

- Every node is placed on the top of previous node.

Example

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.layout.StackPane;
import javafx.scene.control.Button;
import javafx.stage.Stage;
public class myjavafxapplication extends Application {
public void start(Stage stage) {
StackPane root=new StackPane();
Scene scene=new Scene(root,300,250);
Button B1=new Button("One");
Button B2=new Button("Two");
Button B3=new Button("Three");
Button B4=new Button("Four");
Button B5=new Button("Five");
root.getChildren().addAll(B1,B2,B3,B4,B5);
stage.setScene(scene);
stage.setTitle("Layout Demo");
stage.show();
}
public static void main(String[] args) {
launch(args);
}
}
```

Output



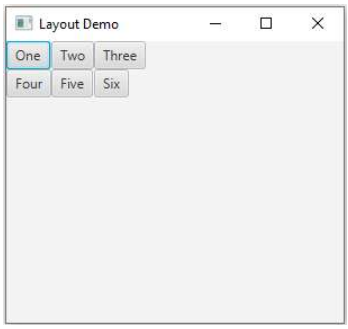
5.4.5 GridPane

- Places nodes into a grid of rows and columns.

Example

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.layout.GridPane;
import javafx.scene.control.Button;
import javafx.stage.Stage;
public class myjavafxapplication extends Application {
public void start(Stage stage) {
GridPane root=new GridPane();
Scene scene=new Scene(root,300,250);
Button B1=new Button("One");
Button B2=new Button("Two");
Button B3=new Button("Three");
Button B4=new Button("Four");
Button B5=new Button("Five");
Button B6=new Button("Six");
root.add(B1,0,0);
root.add(B2,1,0);
root.add(B3,2,0);
root.add(B4,0,1);
root.add(B5,1,1);
root.add(B6,2,1);
stage.setScene(scene);
stage.setTitle("Layout Demo");
stage.show();
}
public static void main(String[] args) {
launch(args);
}
}
```

Output



5.5 Menus

5.5.1 Menu

- Create a menu using Menu class
- Popup menu contains several menu items that are displayed when the user clicks a menu
- User can select a menu item from the menu
- Constructors:
    - Menu()

- Menu(String str)
- Menu(String str,Node nd)
- Menu(String str,Node nd,Items)

5.5.2 MenuBar

- Just like a navigation bar with menus on it
- Located at the top of the screen
- Constructors:
  - MenuBar()
  - MenuBar(menu items)

5.5.3 MenuItems

- Basic item that goes on a menu

Example

```
import javafx.application.Application;
import static javafx.application.Application.launch;
import javafx.scene.Scene;
import javafx.scene.layout.BorderPane;
import javafx.scene.control.Menu;
import javafx.scene.control.MenuBar;
import javafx.scene.control.MenuItem;
import javafx.stage.Stage;
public class myjavafxapplication extends Application {
public void start(Stage stage) {
BorderPane root=new BorderPane();
Scene scene=new Scene(root,300,250);
MenuBar MB=new MenuBar();
Menu m1=new Menu("File");
MenuItem MI1=new MenuItem("New");
MenuItem MI2=new MenuItem("Open");
MenuItem MI3=new MenuItem("Save");
MenuItem MI4=new MenuItem("Exit");
m1.getItems().addAll(MI1,MI2,MI3,MI4);
Menu m2=new Menu("Edit");
MenuItem MI5=new MenuItem("Cut");
MenuItem MI6=new MenuItem("Copy");
MenuItem MI7=new MenuItem("Paste");
m2.getItems().addAll(MI5,MI6,MI7);
Menu m3=new Menu("About");
MB.getMenus().addAll(m1,m2,m3);
root.setTop(MB);
stage.setScene(scene);
stage.setTitle("Layout Demo");
stage.show();
}
public static void main(String[] args) {
launch(args);
}
}
```

Output

